
PROJECT #3 — RANK #3

1. Project Title

CryptoMigrate-Q: An Open-Source Harvest-Now-Decrypt-Later Risk Quantification Engine and Automated Post-Quantum Cryptographic Migration Orchestration Platform

---

2. Problem Statement

Nation-state adversaries are executing **"Harvest Now, Decrypt Later" (HNDL)** operations: systematically collecting today's encrypted network traffic at scale, storing it in cold storage, and waiting for quantum computers powerful enough to break RSA/ECC encryption — estimated 2030–2040.

Critically vulnerable data includes long-lived secrets with:

- Classification lifetime > quantum timeline (classified government data, sealed legal records, proprietary IP)
- Any data encrypted with RSA-2048, ECDH, ECDSA — all broken by Shor's algorithm in polynomial time

Organizations face three critical failures:

1. **No visibility:** They don't know which of their encrypted communications are HNDL-exposed
2. **No risk quantification:** No model combines data sensitivity lifetime × quantum timeline uncertainty → risk score
3. **No migration tooling:** No free, automated tool exists to discover cryptographic assets, prioritize them, and migrate them to NIST PQC standards

NSA, CISA, and NIST have issued warnings since 2021 — but only guidance documents, no open-source tooling.

---

3. Existing Research Summary

HNDL Threat (well-known conceptually, not solved tooling-wise):

- NSA CNSA 2.0 (2022): Timeline for migration to PQC algorithms

- CISA/NSA/NIST Advisory (2022): *Preparing for Post-Quantum Cryptography* — guidance only, no tooling
- Mosca's Theorem (2015): Quantifies migration urgency as X + Y > Z (security shelf-life + migration time > quantum timeline)

Quantum Timeline Estimation:

- Webber et al. 2022 (*The Impact of Hardware Specifications on Reaching Quantum Advantage*): Estimates CRQC timelines
- Babbush et al. 2021 (*Focus Beyond Quadratic Speedups*): Algorithmic analysis of CRQC requirements
- Wide variance: 2030–2045+ depending on error correction progress

Cryptographic Inventory/Discovery (commercial, closed-source):

- PQShield CryptoDiscovery: Commercial tool, $XX,XXX/year
- Rambus Cryptography Research: Enterprise product, not open-source
- No open-source cryptographic asset inventory tool exists with HNDL modeling

PQC Migration Guidance:

- NIST SP 1800-38B (*Migration to Post-Quantum Cryptography*): Technical guidance; manual process
- ETSI TS 103 744: PQC migration guidance; no automation

What has been solved: HNDL concept; PQC algorithm standardization; migration guidance documents.

What has NOT been solved: Open-source cryptographic inventory; automated HNDL risk quantification; code-level migration tooling; quantum timeline simulation for risk modeling.

---

4. Research Gap

Gap	Specific Absence	Industry Impact
No HNDL risk model	No formal model combining data sensitivity, crypto lifetime, quantum timeline uncertainty	Organizations cannot prioritize migration; doing everything equally = doing nothing
No open-source crypto scanner	Commercial tools only; non-profit hospitals, universities, governments	Public sector most exposed, least protected

Gap	Specific Absence	Industry Impact
	cannot afford them	
No automated migration	Current migration requires manual code review; infeasible for large codebases	Migration timeline extends beyond quantum arrival
No quantum timeline uncertainty model	Binary assumption (quantum will/won't arrive) rather than probabilistic distribution	Risk is systematically misestimated
No dual-stack transition bridge	No framework for running classical + PQ crypto simultaneously during migration	Organizations must choose hard cutover (risky) or do nothing (dangerous)

---

5. Proposed Innovation

CryptoMigrate-Q delivers:

1. **HNDL Risk Score Model:** A Bayesian network combining:
    - Data sensitivity lifetime distribution (per asset)
    - Quantum timeline probability distribution (Monte Carlo over expert estimates)
    - Cryptographic exposure score (which primitives, what key sizes)
    - Output: Per-asset HNDL risk score with confidence interval
2. **CryptoDiscovery Engine:** Hybrid static + dynamic scanner:
    - Network layer: nmap-based TLS handshake cipher suite capture
    - Code layer: Tree-sitter AST parser detecting crypto API usage across Python, Java, Go, C/C++
3. **Migration Orchestrator:** Dependency-aware migration graph that generates:
    - Prioritized migration task list (by risk score × complexity)
    - Code diffs (LLM-assisted) for API migration (e.g., RSA → Kyber, ECDSA → Dilithium)
    - Infrastructure-as-Code patches (Terraform/Ansible) for TLS configuration
4. **Hybrid Bridge Module:** Dual-stack implementation allowing classical + PQ simultaneous operation during transition
5. **Quantum Timeline Simulator:** Monte Carlo simulation using CRQC timeline probability distributions from current expert consensus

---

6. Technical Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                     CryptoMigrate-Q ARCHITECTURE                         │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  INPUT LAYER                                                             │
│                                                                          │
│  ┌────────────────────────────────────────────────────────────────────┐ │
│  │   Network Interfaces       Source Code Repos       Config Files     │ │
│  │   (TLS sessions)           (GitHub/GitLab/local)   (nginx, Apache)  │ │
│  └──────────┬────────────────────────┬────────────────────────┬────────┘ │
│             ▼                        ▼                        ▼          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐       │
│  │ NETWORK SCANNER  │  │ CODE ANALYZER    │  │ CONFIG SCANNER   │       │
│  │ nmap TLS probe   │  │ Tree-sitter AST  │  │ Regex pattern    │       │
│  │ cipher capture   │  │ Crypto API detect│  │ matching         │       │
│  └────────┬─────────┘  └────────┬─────────┘  └────────┬─────────┘       │
│           └─────────────────────┼──────────────────────┘                 │
│                                 ▼                                        │
│  ┌───────────────────────────────────────────────────────────────────┐  │
│  │                   CRYPTO ASSET REGISTRY                            │  │
│  │  Asset: {location, algorithm, key_size, usage_context, data_type} │  │
│  └──────────────────────────────┬────────────────────────────────────┘  │
│                                 ▼                                        │
│  ┌───────────────────────────────────────────────────────────────────┐  │
│  │                   HNDL RISK QUANTIFIER                             │  │
│  │                                                                    │  │
```

```
│  │  Bayesian Network:                                        │
│
│  │  • P(CRQC by year T) — Monte Carlo over expert estimates  │
│
│  │  • Data sensitivity lifetime — ML classifier on asset context │
│
│  │  • Quantum vulnerability score — algorithm × key size mapping │
│
│  │  → Risk Score: P(data decrypted before obsolete) ∈ [0,1]  │  │
│  └──────────────────────────┬──────────────────────────────┘
│
│                             ▼
│
│  ┌─────────────────────────────────────────────────────────┐
│
│  │                MIGRATION ORCHESTRATOR                    │
│
│  │                                                          │  │
│  │  ┌──────────────┐    ┌──────────────────────────────┐   │  │
│  │  │ Priority Queue │   │ LLM Code Transformer         │   │  │
│  │  │ (Risk × Effort)│──►│ (RSA→Kyber, ECDSA→Dilithium diffs) │ │  │
│  │  └──────────────┘    └──────────────────────────────┘   │  │
│  │  ┌──────────────┐    ┌──────────────────────────────┐   │  │
│  │  │ IaC Patch Gen. │   │ Hybrid Bridge (Dual-Stack)   │   │  │
│  │  │ (Terraform/Ansible│ │ Classical ⊕ PQ concurrent support │ │
│
│  │  └──────────────┘    └──────────────────────────────┘   │  │
│  └──────────────────────────┬──────────────────────────────┘
│
│                             ▼
│
│  ┌─────────────────────────────────────────────────────────┐
│
│  │          RISK DASHBOARD (Dash/Plotly + React)            │  │
│  │  HNDL Risk posture | Migration progress | Quantum timeline model │ │
│  └─────────────────────────────────────────────────────────┘
│
└─────────────────────────────────────────────────────────────┘
```

Security Model:

- The tool itself processes sensitive crypto inventory data → all local execution, no cloud dependency
- Asset registry encrypted at rest using AES-256 → migrations to Kyber once completed

- Code patches are diff-only, never executing on production without human review

---

7. Quantum Components

Component	Tool	Purpose
Quantum Timeline Model	SciPy Monte Carlo + expert distribution data	Model P(CRQC available by year T) with uncertainty
Shor's Algorithm Complexity Estimator	Python + published CRQC resource estimates	Map key sizes to qubit/gate requirements for break
PQ Migration Targets	liboqs + PQClean	Source of NIST-standard Kyber/Dilithium/FALCON implementations
Hybrid Bridge	liboqs + OpenSSL (oqs-provider)	Simultaneous classical + PQ TLS during transition
QUBO Optimization	D-Wave Ocean SDK (free tier) or PyQUBO	Optional: Frame migration scheduling as quantum optimization

---

8. Cybersecurity Components

- **Cryptographic Agility:** AST-based detection of all crypto API usage across 5 languages
- **TLS Analysis:** Cipher suite enumeration, certificate chain analysis, key size extraction
- **NIST PQC Standards:** Integration of ML-KEM, ML-DSA, SLH-DSA (SPHINCS+)
- **Risk Modeling:** Bayesian quantification of HNDL attack probability over time
- **Secure Development:** SAST-style scanning for cryptographic vulnerabilities in source code

---

9. AI Components

Module	Architecture	Purpose
Data Sensitivity Classifier	Fine-tuned BERT/DistilBERT	Classify data type (medical, financial, government, PII) from asset context/comments

Module	Architecture	Purpose
Risk Score Regressor	Gradient Boosted (XGBoost)	Predict HNDL risk score from asset features
Code Transformer	LLM API (Ollama local model)	Generate code diffs for crypto API migration
Timeline Simulator	Bayesian Network (pgmpy)	Monte Carlo CRQC timeline modeling
Dependency Graph Analyzer	NetworkX + GNN	Identify crypto dependency chains for safe migration ordering

---

10. GitHub Repositories

Repository	Purpose	Link	Project Contribution
open-quantum-safe/liboqs	PQC implementations	github.com/open-quantum-safe/liboqs	Migration target algorithms (Kyber, Dilithium, FALCON)
open-quantum-safe/oqs-provider	OpenSSL PQ integration	github.com/open-quantum-safe/oqs-provider	Hybrid classical+PQ TLS bridge
PQClean/PQClean	Clean PQC code	github.com/PQClean/PQClean	Reference implementations for new crypto target
tree-sitter/tree-sitter	AST parser	github.com/tree-sitter/tree-sitter	Multi-language crypto API detection
networkx/networkx	Graph library	github.com/networkx/networkx	Crypto dependency graph analysis

Repository	Purpose	Link	Project Contribution
pgmpy/pgmpy	Bayesian networks	github.com/pgmpy/pgmpy	HNDL risk Bayesian model
dwave-examples	QUBO optimization	github.com/dwave-examples	Optional quantum-optimized migration scheduling
plotly/dash	Dashboard framework	github.com/plotly/dash	HNDL risk posture visualization
ollama/ollama	Local LLM runner	github.com/ollama/ollama	Code migration diff generation without cloud API

---

11. Development Roadmap (12 Months)

Phase 1 — Risk Model (Months 1–2): Design Bayesian HNDL risk model; collect CRQC timeline data from published papers; implement Monte Carlo simulator

Phase 2 — Crypto Discovery (Months 3–5): Build Tree-sitter AST scanner for Python, Java, Go, C; implement nmap TLS cipher capture; build asset registry schema + SQLite storage

Phase 3 — Risk Engine (Months 6–7): Integrate discovery + risk model; train data sensitivity classifier on labeled asset dataset; build priority queue logic

Phase 4 — Migration Orchestrator (Months 8–9): Integrate Ollama-based code transformer for diff generation; build IaC patch generator; implement hybrid bridge using oqs-provider

Phase 5 — Dashboard + Evaluation (Months 10–12): Build Dash dashboard; evaluate on open-source project codebases (OpenSSL, OpenSSH) as test targets; write IEEE paper

Agentic Development Breakdown:

- **Claude Code:** AST scanner, risk model implementation, migration diff logic
- **Cursor:** Dash dashboard, REST API, database schema
- **OpenHands:** Automated scanning pipeline, Docker containerization, test suite generation
- **Gemini CLI:** Documentation generation, paper outline, evaluation scripts

12. Publication Potential

Venue	Tier	Justification
IEEE S&P (Security and Privacy)	Top-1	Novel HNDL risk model + first open-source tooling
USENIX Security	Top-1	Systems tool + empirical evaluation on real codebases
IEEE Transactions on Dependable and Secure Computing	Q1 Journal	Applied PQC migration research
ACM CCS	Top-1	Cryptographic engineering + systems security focus

13. Scoring Table

Criterion	Score	Justification
IEEE Publication Potential	**9/10**	USENIX/S&P tier; fills massive tooling gap
Research Novelty	**9/10**	First open-source HNDL risk + migration platform
Industry Relevance 2026–2035	**10/10**	HNDL is active NOW; NIST migration deadline 2030
Quantum Depth	**8/10**	Deep quantum threat modeling; CRQC timeline simulation; Shor's complexity analysis
Cybersecurity Depth	**9/10**	Cryptographic agility + SAST + Bayesian risk + PQC migration
Open Source Feasibility	**9/10**	Entirely free stack; runs locally on any laptop
Agent-First Development	**10/10**	Code scanning, risk modeling, migration diffs — perfectly suited to agentic coding
TOTAL	**64/70**	